



Igneous Inf 1.5

Security Assessment

February 12th, 2026 — Prepared by OtterSec

Tamta Topuria

tamta@osec.io

Artem Astakhov

aikasimirov@osec.io

Table of Contents

Executive Summary	2
Overview	2
Key Findings	2
Scope	3
Findings	4
Vulnerabilities	5
OS-IGN-ADV-00 LST Flash Loan Reward Exploit	6
OS-IGN-ADV-01 LST Transfer Fee Handling	7
General Findings	8
OS-IGN-SUG-00 Admin Key Misconfiguration Lockout	9
OS-IGN-SUG-01 Misleading Argument Type	10
Appendices	
Vulnerability Rating Scale	11
Procedure	12

01 — Executive Summary

Overview

Igneous labs engaged OtterSec to assess the `s-controller-v2` program. This assessment was conducted between January 7th and February 10th, 2026. For more information on our auditing methodology, refer to [Appendix B](#).

Key Findings

We produced 4 findings throughout this audit engagement.

In particular, we identified a vulnerability where calculators do not enforce epoch-based update checks, enabling a user to exploit LST programs by updating rewards mid-transaction to redeem at inflated SOL values, profiting from accrued rewards without real staking ([OS-IGN-ADV-00](#)). Additionally, an LST with TransferFee or TransferHook extensions may result in the pool receiving less than expected or users getting fewer tokens than calculated, breaking slippage checks and pool functions ([OS-IGN-ADV-01](#)).

We also advised verifying that the new key is controlled by its intended owner, since otherwise, administrative instructions risk permanent lockout if a new admin or fee beneficiary key is entered incorrectly ([OS-IGN-SUG-00](#)). We further suggested utilizing the correct argument type to avoid confusion and improve clarity ([OS-IGN-SUG-01](#)).

02 — Scope

The source code was delivered to us in a Git repository at <https://github.com/igneous-labs/inf-1.5>. This audit was performed against commit [5e6cbfa](#).

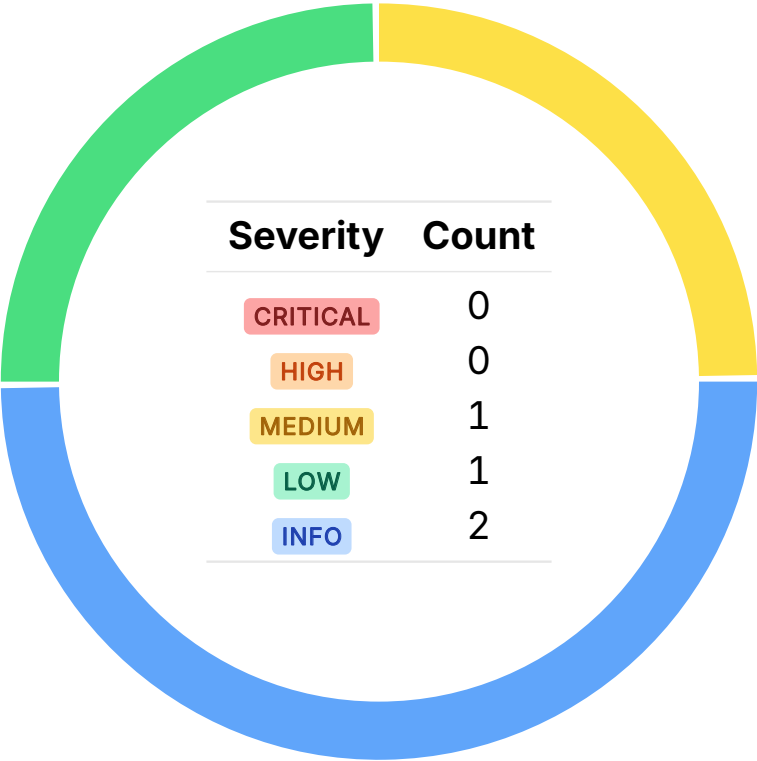
A brief description of the program is as follows:

Name	Description
s-controller-v2	The INF controller program is the core Solana program of the INF liquid staking protocol. It manages a unified liquidity pool that holds multiple Liquid Staking Tokens (LSTs), enabling users to swap between them and add or remove liquidity. It tracks each LST’s SOL value via external calculator programs, gradually releases accumulated yield to liquidity providers through a Release-Per-Slot mechanism, and collects protocol fees. Admins can configure LSTs, rebalance reserves, and disable the pool in emergencies.

03 — Findings

Overall, we reported 4 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in [Appendix A](#).

ID	Severity	Status	Description
OS-IGN-ADV-00	MEDIUM	RESOLVED ✓	In cases where calculators do not enforce epoch-based update checks, a user may exploit LST programs by updating rewards mid-transaction to redeem at inflated SOL values, profiting from accrued rewards without real staking.
OS-IGN-ADV-01	LOW	RESOLVED ✓	An LST with <code>TransferFee</code> or <code>TransferHook</code> extensions may result in the pool receiving less than expected or users getting fewer tokens than calculated, breaking slippage checks and pool functions.

LST Flash Loan Reward Exploit MEDIUM

OS-IGN-ADV-00

Description

It is possible for a user to atomically trigger a reward update in an LST protocol and immediately redeem at the newly increased exchange rate. If the SOL value calculator does not enforce that the pool's exchange rate has already been updated in the current epoch, a user may bundle an update instruction with LST redemption in the same transaction. However, this attack is profitable only if the pricing program's fees are lower than the reward-induced price increase.

Proof of Concept

The following steps illustrate the above scenario:

1. A user obtains a large amount of `LST1` via flash loan to maximize extractable reward delta.
2. The user swaps `LST1` into `LST2` with Sanctum's Controller program.
3. The user invokes the `LST2` protocol's update instruction (such as `update_active` in Marinade). This realizes accumulated staking rewards, increasing `sol_balance`, which increases the exchange rate for `LST2`.
4. The attacker immediately swaps `LST2` → `LST1` via Sanctum's Controller, gaining a profit due to the newly updated exchange rate.
5. Consequently, the attacker repays the original `LST1` borrowed in the flash loan and retains the profit.

Remediation

Ensure that all calculator programs verify that an update has occurred in the current epoch before computing `lst_to_sol` or `sol_to_lst`. For reference, Lido's calculator program includes this check (`self.computed_in_epoch == this_epoch`), which is absent in Marinade's implementation.

Patch

Team mitigated the issue by setting higher fees for the affected LST in the pricing program than the attacker's potential gain.

LST Transfer Fee Handling LOW

OS-IGN-ADV-01

Description

There is a potential issue when an LST has a `TransferFee` or `TransferHook` extension, which deducts tokens during transfers. In such a scenario, if the LST is an input token, the pool receives less than the expected amount, resulting in functions such as `final_sync` failing and rendering the LST unusable in the pool. Conversely, if the LST is an output token, the user receives fewer tokens than calculated, because slippage checks are performed before fees are deducted. The core problem is that the Controller program assumes full-amount transfers, so any post-transfer deductions break these assumptions.

Remediation

Disallow LST with a `TransferFee` or `TransferHook` extension.

Patch

The team will ensure, off-chain, that LSTs with `TransferFee` or `TransferHook` extensions are not added.

05 — General Findings

Here, we present a discussion of general findings identified during our audit. While these findings do not pose an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-IGN-SUG-00	Administrative instructions risk permanent lockout if a new admin or fee beneficiary key is entered incorrectly, since the current checks do not verify that the new key is controlled by its intended owner.
OS-IGN-SUG-01	The current code utilizes <code>PriceExactOutIxArgs</code> for a <code>PriceExactIn</code> CPI, which is misleading and affects clarity.

Admin Key Misconfiguration Lockout

OS-IGN-SUG-00

Description

There is a possible lockout risk in administrative instructions such as `set_admin` and `set_protocol_fee_beneficiary`. Currently, these instructions only require the current admin or authority to sign, so if the new admin or new fee beneficiary key is entered incorrectly (due to a typo), the pool will become permanently inaccessible.

Remediation

Require the new admin to also be a signer in `set_admin`, ensuring the key is valid and controlled by the intended party. For `set_protocol_fee_beneficiary`, the signer should be allowed to be either the admin or the current `fee_beneficiary`, similar to the `set_rps_auth` check, enabling the admin to correct mistakes.

Patch

Resolved in [13dcc61](#).

Misleading Argument Type

OS-IGN-SUG-01

Description

In the current implementation of `exact_in`, the call to `cpi_price_exact_in` in `exec_calc_cpis_unchecked` is invoked with `PriceExactOutIxArgs`, which compiles only because `PriceExactInIxArgs` and `PriceExactOutIxArgs` are identical type aliases. Although structurally compatible, the names represent different swap semantics. Utilizing the “exact out” type in an “exact in” flow obscures developer intent and reduces code clarity.

```
>_ src/instructions/swap/v2/exact_in.rs
```

RUST

```
fn exec_calc_cpis_unchecked(
    abr: &mut Abr,
    cpi: &mut Cpi,
    accs: &SwapV2CtlIxAccounts,
    amount: u64,
) -> Result<SwapCpiRetVals, ProgramError> {
    [...]
    let out_sol_val = cpi_price_exact_in(
        cpi,
        abr,
        pricing_prog,
        PriceExactOutIxArgs {
            amt: amount,
            sol_value: inp_sol_value,
        },
        &PriceExactOutIxAccountHandles {
            ix_prefix: NewPpIxPreAccsBuilder::start()
                .with_input_mint(*ix_prefix.inp_mint())
                .with_output_mint(*ix_prefix.out_mint())
                .build(),
            suf: pricing,
        },
    )?;
    [...]
}
```

Remediation

Utilize `PriceExactInIxArgs` when calling `cpi_price_exact_in`.

Patch

Resolved in [2996ef9](#)

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the [General Findings](#).

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
 - Improperly designed economic incentives leading to loss of funds.
-

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
 - Exploitation involving high capital requirement with respect to payout.
-

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
 - Forced exceptions in the normal user flow.
-

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.
-

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
 - Improved input validation.
-

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.